Deep Reinforcement Learning

Unit 8: Proximal Policy Optimization (PPO)

Hugging Face Deep RL Course

Complete MDX-to-LaTeX study edition

Source: Hugging Face Deep RL Course (original English text).

Contents

1 Introduction **2**

2 The intuition behind PPO **3**

3 Introducing the Clipped Surrogate Objective Function **4**
- 3.1 Recap: The Policy Objective Function . . . 4
- 3.2 The Ratio Function . . . 5
- 3.3 The unclipped part of the Clipped Surrogate Objective function . . . 6
- 3.4 The clipped Part of the Clipped Surrogate Objective function . . . 7

4 Visualize the Clipped Surrogate Objective Function **8**
- 4.1 Case 1 and 2: the ratio is between the range . . . 9
- 4.2 Case 3 and 4: the ratio is below the range . . . 9
- 4.3 Case 5 and 6: the ratio is above the range . . . 10

5 Hands-on **11**
- 5.1 Unit 8: Proximal Policy Gradient (PPO) with PyTorch . . . 12
- 5.2 Objectives of this notebook . . . 12
- 5.3 Prerequisites . . . 13
- 5.4 Set the GPU . . . 13
- 5.5 Create a virtual display . . . 14
- 5.6 Install dependencies . . . 14
- 5.7 Let's code PPO from scratch with Costa Huang's tutorial . . . 14
- 5.8 Add the Hugging Face Integration . . . 15
- 5.9 Let's start the training . . . 29
- 5.10 Some additional challenges . . . 30
- 5.11 Keep learning, stay awesome . . . 31

6 Conclusion **31**
- 6.1 Keep Learning, Stay awesome . . . 31

7 Additional Readings **31**
- 7.1 PPO Explained . . . 31
- 7.2 PPO Implementation details . . . 31
- 7.3 Importance Sampling . . . 31

8 Introduction to PPO with Sample-Factory **31**

9 Hands-on: advanced Deep Reinforcement Learning. Using Sample Factory to play Doom from pixels **32**
- 9.1 Unit 8 Part 2: Advanced Deep Reinforcement Learning. Using Sample Factory to play Doom from pixels . . . 33

9.2 Set the GPU . . . 34
9.3 Sample Factory . . . 34
9.4 ViZDoom . . . 36
9.5 We first need to install some dependencies that are required for the ViZDoom environment . . . 37
9.6 Then we can install Sample Factory and ViZDoom . . . 37
9.7 Setting up the Doom Environment in sample-factory . . . 37
9.8 Training the agent . . . 39
9.9 Let's take a look at the performance of the trained policy and output a video of the agent. . . . 39
9.10 Now lets visualize the performance of the agent . . . 39
9.11 Now lets upload your checkpoint and video to the Hugging Face Hub . . . 39
9.12 Let's load another model . . . 40
9.13 Some additional challenges : Doom Deathmatch . . . 41
9.14 Keep learning, stay awesome . . . 42

10 Conclusion **42**
10.1 Keep Learning, Stay awesome . . . 42

1 Introduction

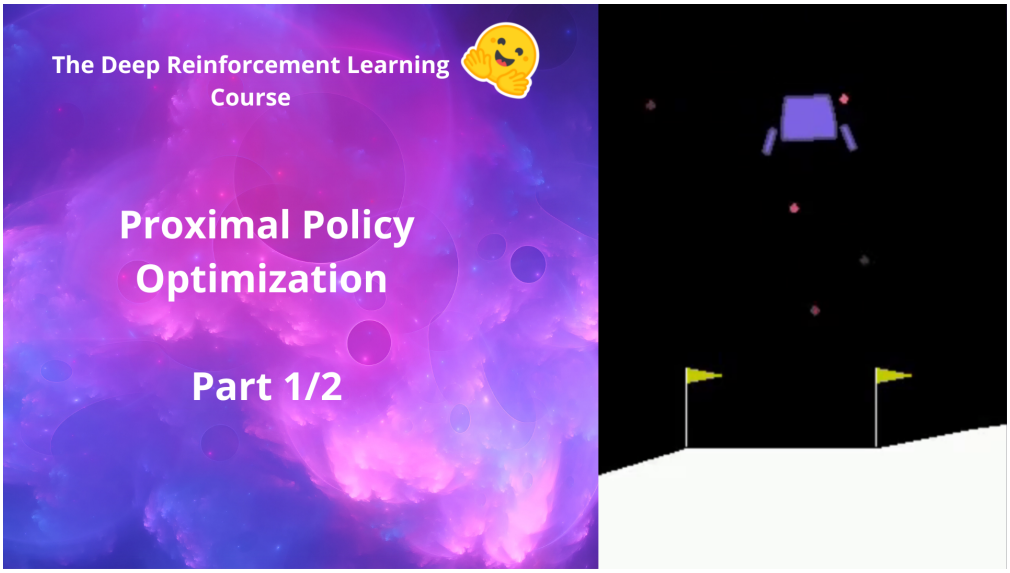


Figure 1: Hugging Face Deep RL Course, Unit 8: PPO [original figure]

In Unit 6, we learned about Advantage Actor Critic (A2C), a hybrid architecture combining value-based and policy-based methods that helps to stabilize the training by reducing the variance with:

- *An Actor* that controls **how our agent behaves** (policy-based method).
- *A Critic* that measures **how good the action taken is** (value-based method).

Today we'll learn about Proximal Policy Optimization (PPO), an architecture that **improves our agent's training stability by avoiding policy updates that are too large**. To do that, we use a ratio that indicates the difference between our current and old policy and clip this ratio to a specific range $[1 - \epsilon, 1 + \epsilon]$.

Doing this will ensure **that our policy update will not be too large and that the training is more stable.**

This Unit is in two parts:

- In this first part, you'll learn the theory behind PPO and code your PPO agent from scratch using the CleanRL implementation. To test its robustness you'll use LunarLander-v2. LunarLander-v2 **is the first environment you used when you started this course**. At that time, you didn't know how PPO worked, and now, **you can code it from scratch and train it. How incredible is that** .
- In the second part, we'll get deeper into PPO optimization by using Sample-Factory and train an agent playing vizdoom (an open source version of Doom).



Figure 2: These are the environments you're going to use to train your agents: VizDoom environments [original figure]

Sound exciting? Let's get started!

2 The intuition behind PPO

The idea with Proximal Policy Optimization (PPO) is that we want to improve the training stability of the policy by limiting the change you make to the policy at each training epoch: **we want to avoid having too large of a policy update.**

For two reasons:

- We know empirically that smaller policy updates during training are **more likely to converge to an optimal solution.**
- A too-big step in a policy update can result in falling "off the cliff" (getting a bad policy) **and taking a long time or even having no possibility to recover.**



Figure 3: Taking smaller policy updates to improve the training stability; Modified version from RL – Proximal Policy Optimization (PPO) Explained by Jonathan Hui (https://jonathan-hui.medium.com/rl-proximal-policy-optimization-ppo-explained-77f014ec3f12) [original figure]

So with PPO, we update the policy conservatively. To do so, we need to measure how much

the current policy changed compared to the former one using a ratio calculation between the current and former policy. And we clip this ratio in a range $[1 - \epsilon, 1 + \epsilon]$, meaning that we **remove the incentive for the current policy to go too far from the old one (hence the proximal policy term).**

3 Introducing the Clipped Surrogate Objective Function

3.1 Recap: The Policy Objective Function

Let's remember what the objective is to optimize in Reinforce:

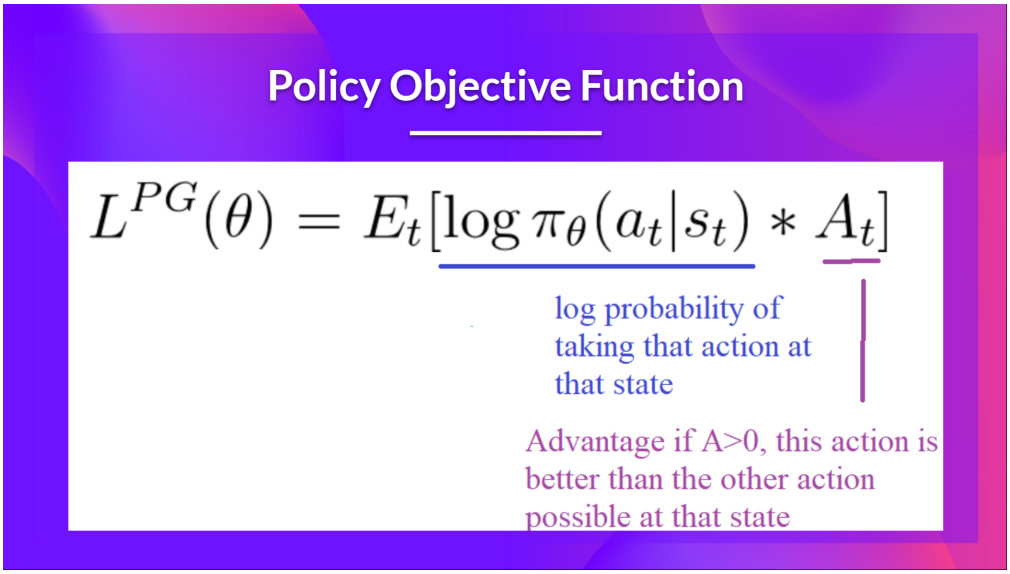


Figure 4: REINFORCE policy-gradient objective [original figure]

The idea was that by taking a gradient ascent step on this function (equivalent to taking gradient descent of the negative of this function), we would **push our agent to take actions that lead to higher rewards and avoid harmful actions.**

However, the problem comes from the step size:

- Too small, **the training process was too slow**
- Too high, **there was too much variability in the training**

With PPO, the idea is to constrain our policy update with a new objective function called the *Clipped surrogate objective function* that **will constrain the policy change in a small range using a clip.**

This new function **is designed to avoid destructively large weights updates** :

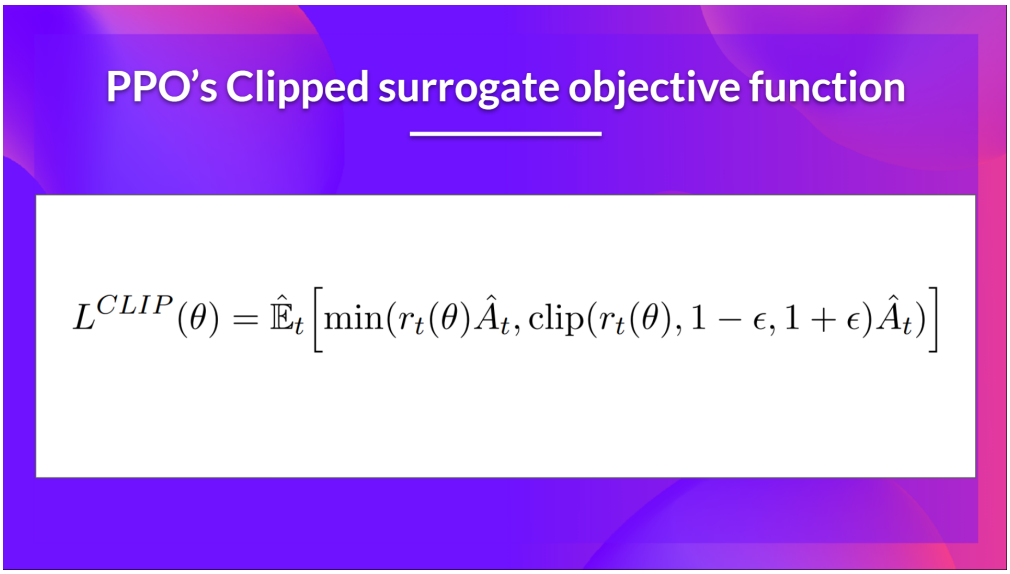


Figure 5: PPO clipped surrogate objective function [original figure]

Let's study each part to understand how it works.

3.2 The Ratio Function

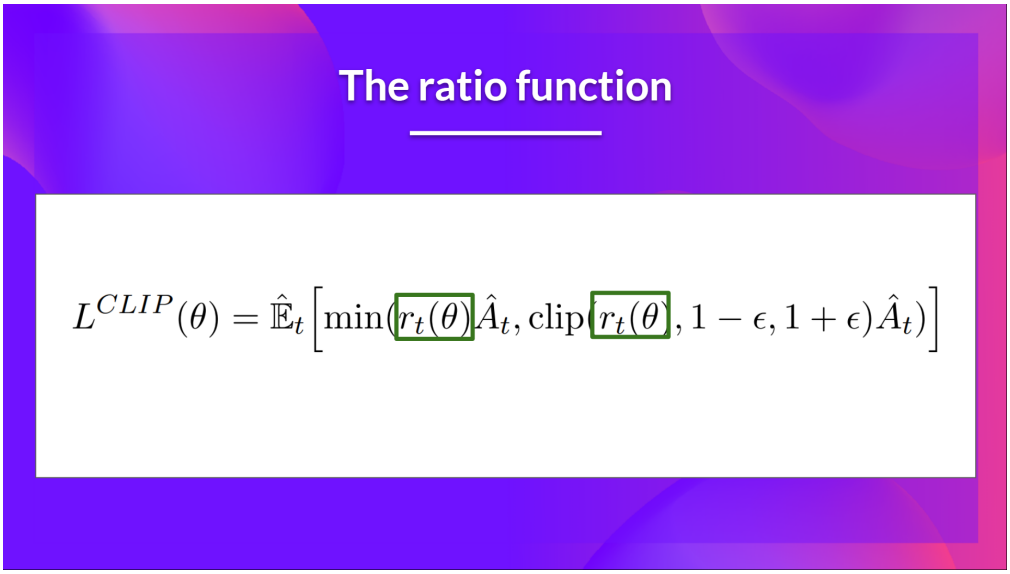


Figure 6: Probability ratio between the old and new policies [original figure]

This ratio is calculated as follows:

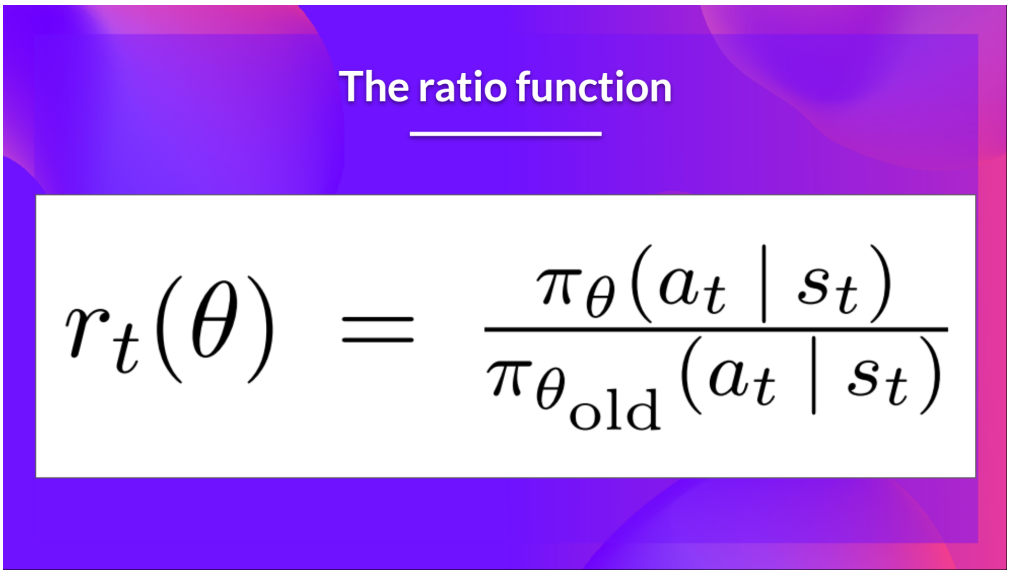


Figure 7: Definition of the PPO policy probability ratio [original figure]

It's the probability of taking action a_t at state s_t in the current policy, divided by the same for the previous policy.

As we can see, $r_t(\theta)$ denotes the probability ratio between the current and old policy:

- If $r_t(\theta) > 1$, the **action a_t at state s_t is more likely in the current policy than the old policy.**
- If $r_t(\theta)$ is between 0 and 1, the **action is less likely for the current policy than for the old one**.

So this probability ratio is an **easy way to estimate the divergence between old and current policy.**

3.3 The unclipped part of the Clipped Surrogate Objective function

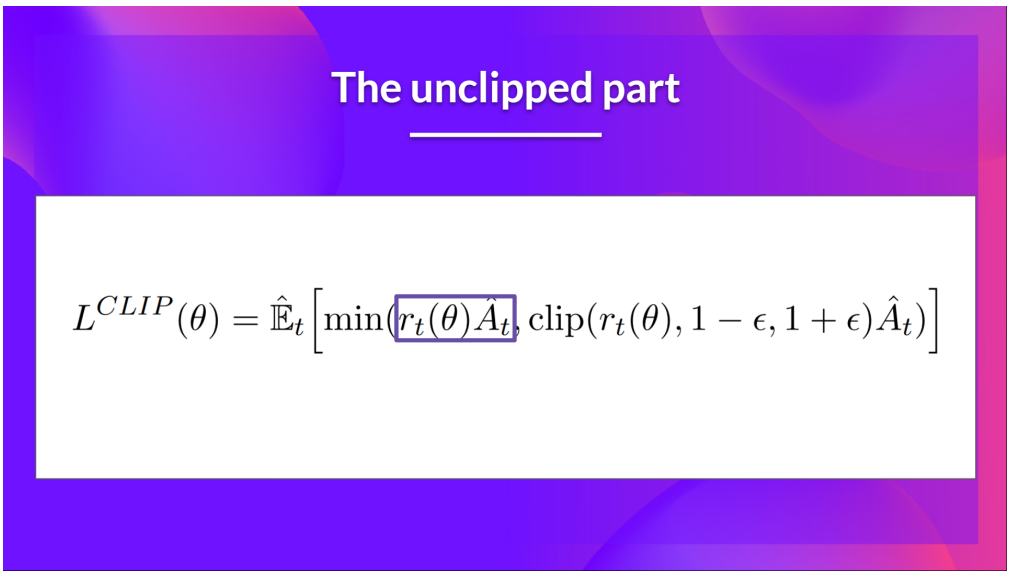


Figure 8: Unclipped surrogate objective of PPO [original figure]

This ratio **can replace the log probability we use in the policy objective function**. This gives us the left part of the new objective function: multiplying the ratio by the advantage.

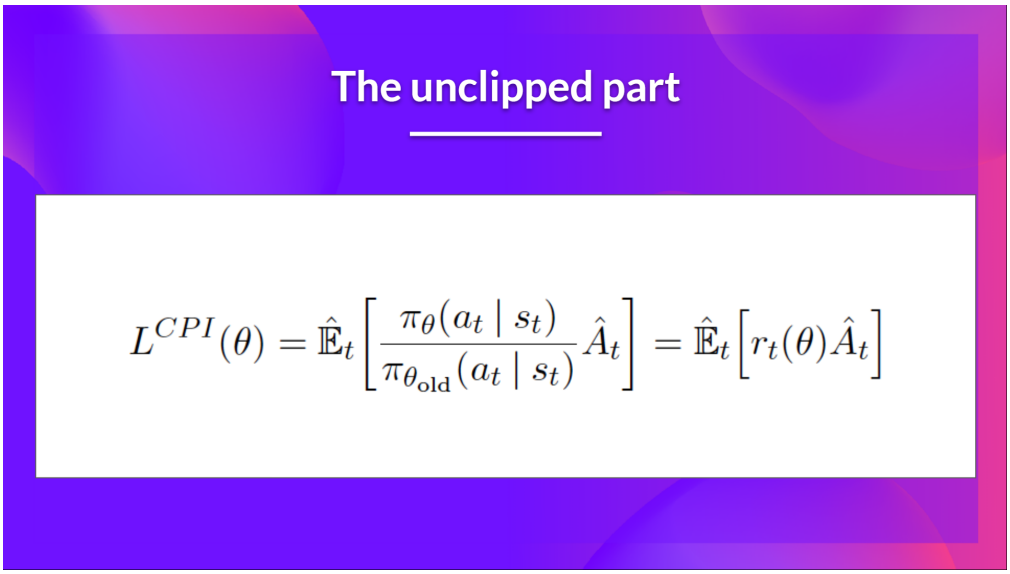


Figure 9: Proximal Policy Optimization Algorithms (https://arxiv.org/pdf/1707.06347.pdf) [original figure]

However, without a constraint, if the action taken is much more probable in our current policy than in our former, **this would lead to a significant policy gradient step** and, therefore, an **excessive policy update.**

3.4 The clipped Part of the Clipped Surrogate Objective function

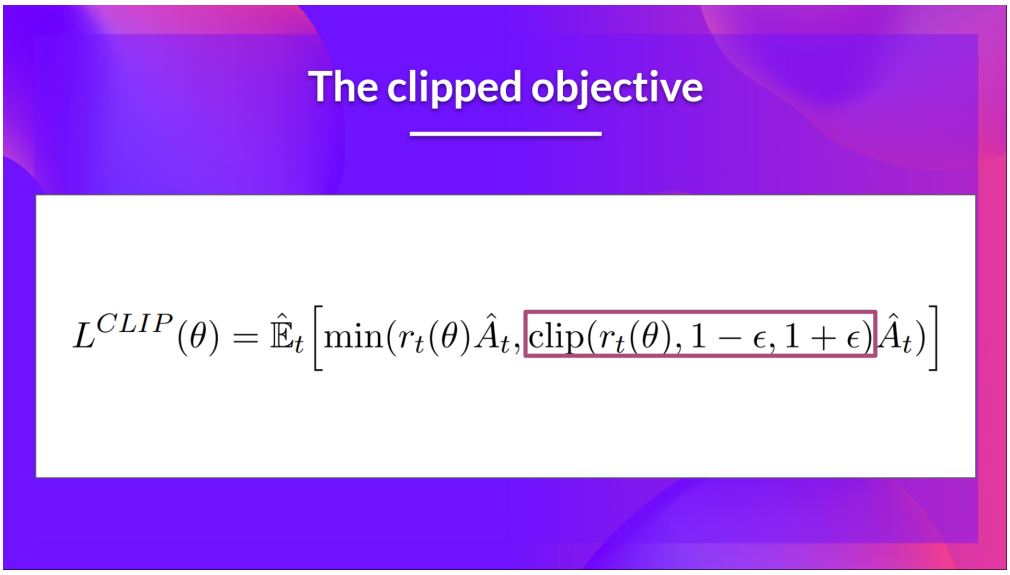


Figure 10: Clipping the probability ratio in PPO [original figure]

Consequently, we need to constrain this objective function by penalizing changes that lead to a ratio far away from 1 (in the paper, the ratio can only vary from 0.8 to 1.2).

By clipping the ratio, we ensure that we do not have a too large policy update because the current policy can't be too different from the older one.

To do that, we have two solutions:

- *TRPO (Trust Region Policy Optimization)* uses KL divergence constraints outside the objective function to constrain the policy update. But this method **is complicated to implement and takes more computation time.**
- *PPO* clip probability ratio directly in the objective function with its **Clipped surrogate objective function.**

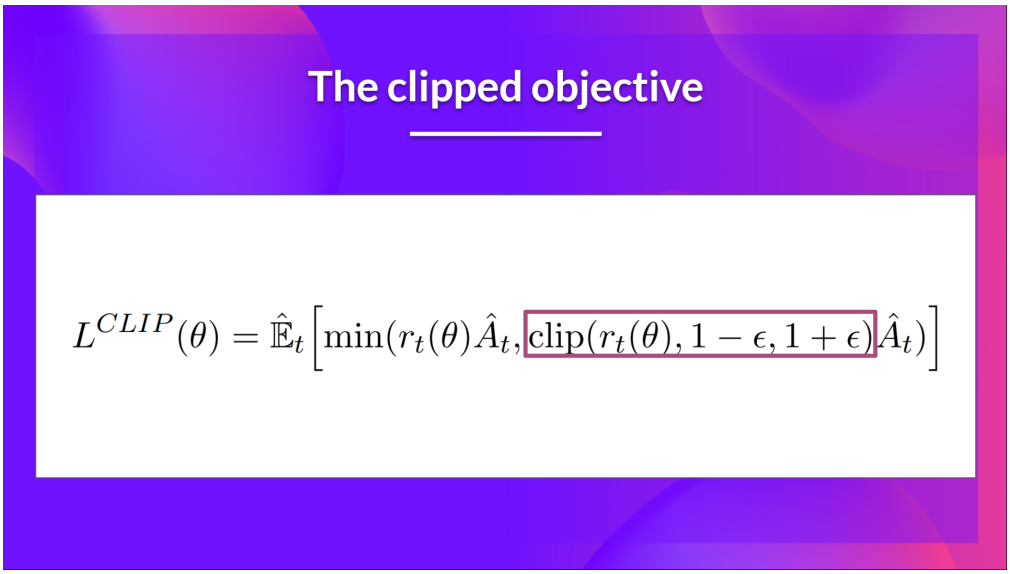


Figure 11: Clipping the probability ratio in PPO [original figure]

This clipped part is a version where $r_t(\theta)$ is clipped between $[1-\epsilon, 1+\epsilon]$.

With the Clipped Surrogate Objective function, we have two probability ratios, one non-clipped and one clipped in a range between $[1-\epsilon, 1+\epsilon]$, epsilon is a hyperparameter that helps us to define this clip range (in the paper $\epsilon = 0.2$.).

Then, we take the minimum of the clipped and non-clipped objective, **so the final objective is a lower bound (pessimistic bound) of the unclipped objective.**

Taking the minimum of the clipped and non-clipped objective means **we'll select either the clipped or the non-clipped objective based on the ratio and advantage situation**.

4 Visualize the Clipped Surrogate Objective Function

Don't worry. **It's normal if this seems complex to handle right now**. But we're going to see what this Clipped Surrogate Objective Function looks like, and this will help you to visualize better what's going on.

	$p_t(\theta)$ > 0	A_t	Return Value of *min*	Objective is Clipped	Sign of Objective	Gradient
1	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	$+$	$p_t(\theta)A_t$	no	$+$	✓
2	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	$-$	$p_t(\theta)A_t$	no	$-$	✓
3	$p_t(\theta) < 1-\epsilon$	$+$	$p_t(\theta)A_t$	no	$+$	✓
4	$p_t(\theta) < 1-\epsilon$	$-$	$(1-\epsilon)A_t$	yes	$-$	**0**
5	$p_t(\theta) > 1+\epsilon$	$+$	$(1+\epsilon)A_t$	yes	$+$	**0**
6	$p_t(\theta) > 1+\epsilon$	$-$	$p_t(\theta)A_t$	no	$-$	✓

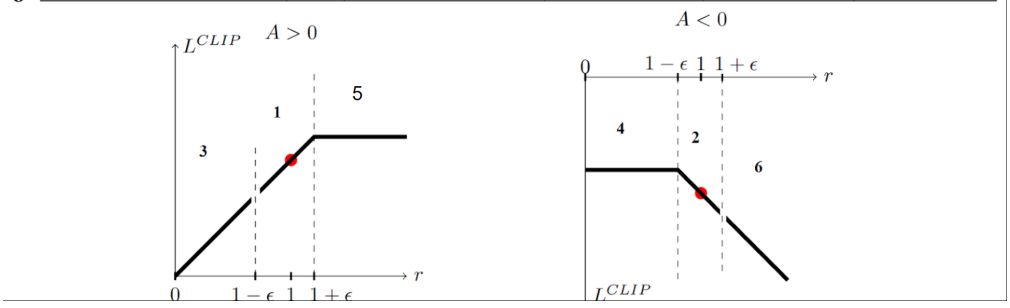


Figure 12: Table from "Towards Delivering a Coherent Self-Contained Explanation of Proximal Policy Optimization" by Daniel Bick (https://fse.studenttheses.ub.rug.nl/25709/1/mAI_2021_BickD.pdf) [original figure]

We have six different situations. Remember first that we take the minimum between the clipped and

unclipped objectives.

4.1 Case 1 and 2: the ratio is between the range

In situations 1 and 2, **the clipping does not apply since the ratio is between the range** $[1-\epsilon, 1+\epsilon]$

In situation 1, we have a positive advantage: the **action is better than the average** of all the actions in that state. Therefore, we should encourage our current policy to increase the probability of taking that action in that state.

Since the ratio is between intervals, **we can increase our policy's probability of taking that action at that state.**

In situation 2, we have a negative advantage: the action is worse than the average of all actions at that state. Therefore, we should discourage our current policy from taking that action in that state.

Since the ratio is between intervals, **we can decrease the probability that our policy takes that action at that state.**

4.2 Case 3 and 4: the ratio is below the range

	$p_t(\theta) > 0$	A_t	**Return Value of** min	**Objective is Clipped**	**Sign of Objective**	**Gradient**
1	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	+	$p_t(\theta)A_t$	no	+	✓
2	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	−	$p_t(\theta)A_t$	no	−	✓
3	$p_t(\theta) < 1-\epsilon$	+	$p_t(\theta)A_t$	no	+	✓
4	$p_t(\theta) < 1-\epsilon$	−	$(1-\epsilon)A_t$	yes	−	**0**
5	$p_t(\theta) > 1+\epsilon$	+	$(1+\epsilon)A_t$	yes	+	**0**
6	$p_t(\theta) > 1+\epsilon$	−	$p_t(\theta)A_t$	no	−	✓

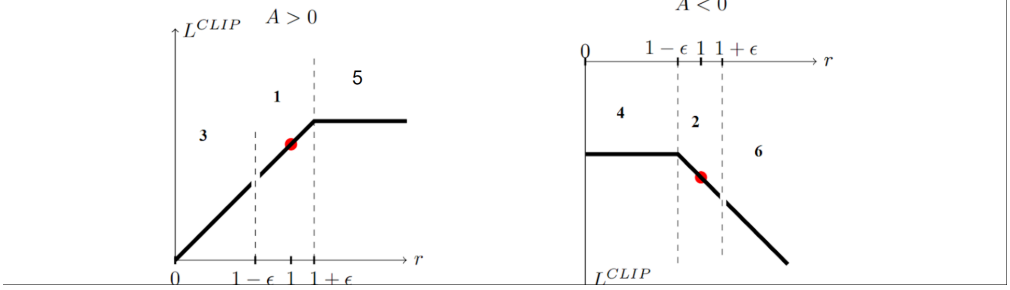


Figure 13: Table from "Towards Delivering a Coherent Self-Contained Explanation of Proximal Policy Optimization" by Daniel Bick (https://fse.studenttheses.ub.rug.nl/25709/1/mAI_2021_BickD.pdf) [original figure]

If the probability ratio is lower than $[1-\epsilon]$, the probability of taking that action at that state is much lower than with the old policy.

If, like in situation 3, the advantage estimate is positive (A>0), then **you want to increase the probability of taking that action at that state.**

But if, like situation 4, the advantage estimate is negative, **we don't want to decrease further** the probability of taking that action at that state. Therefore, the gradient is = 0 (since we're on a flat line), so we don't update our weights.

4.3 Case 5 and 6: the ratio is above the range

	$p_t(\theta) > 0$	A_t	Return Value of min	Objective is Clipped	Sign of Objective	Gradient
1	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	+	$p_t(\theta)A_t$	no	+	✓
2	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	−	$p_t(\theta)A_t$	no	−	✓
3	$p_t(\theta) < 1-\epsilon$	+	$p_t(\theta)A_t$	no	+	✓
4	$p_t(\theta) < 1-\epsilon$	−	$(1-\epsilon)A_t$	yes	−	**0**
5	$p_t(\theta) > 1+\epsilon$	+	$(1+\epsilon)A_t$	yes	+	**0**
6	$p_t(\theta) > 1+\epsilon$	−	$p_t(\theta)A_t$	no	−	✓

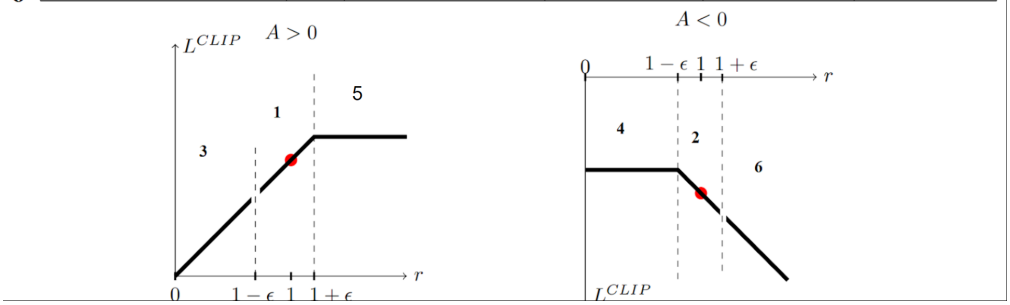


Figure 14: Table from "Towards Delivering a Coherent Self-Contained Explanation of Proximal Policy Optimization" by Daniel Bick (https://fse.studenttheses.ub.rug.nl/25709/1/mAI_2021_BickD.pdf) [original figure]

If the probability ratio is higher than $[1+\epsilon]$, the probability of taking that action at that state in the current policy is **much higher than in the former policy.**

If, like in situation 5, the advantage is positive, **we don't want to get too greedy**. We already have a higher probability of taking that action at that state than the former policy. Therefore, the gradient is $= 0$ (since we're on a flat line), so we don't update our weights.

If, like in situation 6, the advantage is negative, we want to decrease the probability of taking that action at that state.

So if we recap, **we only update the policy with the unclipped objective part**. When the minimum is the clipped objective part, we don't update our policy weights since the gradient will equal 0.

So we update our policy only if:

- Our ratio is in the range $[1-\epsilon, 1+\epsilon]$
- Our ratio is outside the range, but **the advantage leads to getting closer to the range**
  - Being below the ratio but the advantage is > 0
  - Being above the ratio but the advantage is < 0

You might wonder why, when the minimum is the clipped ratio, the gradient is 0. When the ratio is clipped, the derivative in this case will not be the derivative of the $r_t(\theta) * A_t$ but the derivative of either $(1-\epsilon) * A_t$ or the derivative of $(1+\epsilon) * A_t$ which both $= 0$.

To summarize, thanks to this clipped surrogate objective, **we restrict the range that the current policy can vary from the old one.** Because we remove the incentive for the probability ratio to move outside of the interval since the clip forces the gradient to be zero. If the ratio is $> 1+\epsilon$ or $< 1-\epsilon$ the gradient will be equal to 0.

The final Clipped Surrogate Objective Loss for PPO Actor-Critic style looks like this, it's a combination of Clipped Surrogate Objective function, Value Loss Function and Entropy bonus:

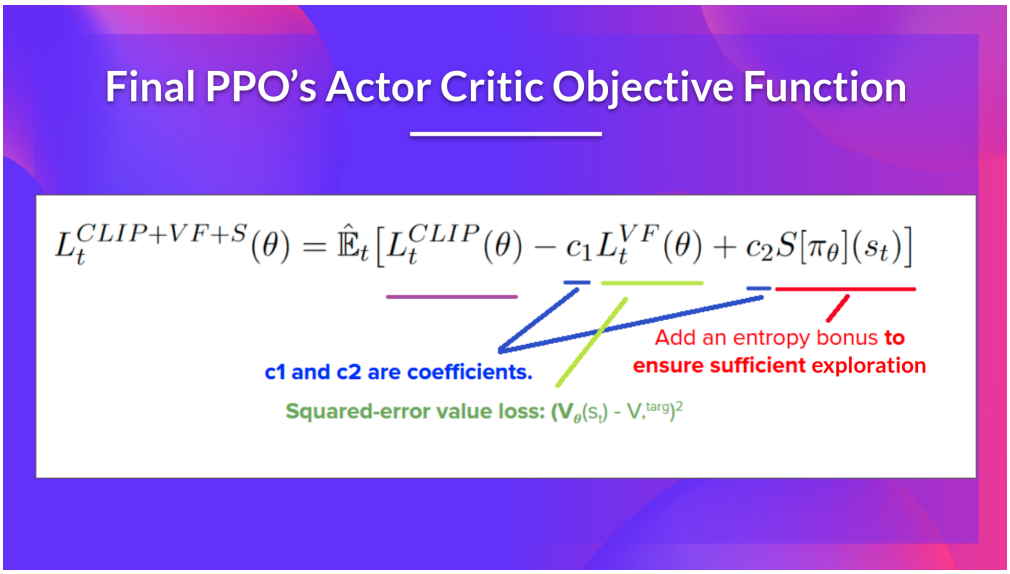


Figure 15: PPO actor-critic objective: policy, value and entropy terms [original figure]

That was quite complex. Take time to understand these situations by looking at the table and the graph. **You must understand why this makes sense.** If you want to go deeper, the best resource is the article "Towards Delivering a Coherent Self-Contained Explanation of Proximal Policy Optimization" by Daniel Bick, especially part 3.4.

5 Hands-on

Original Google Colab notebook

Now that we studied the theory behind PPO, the best way to understand how it works **is to implement it from scratch.**

Implementing an architecture from scratch is the best way to understand it, and it's a good habit. We have already done it for a value-based method with Q-Learning and a Policy-based method with Reinforce.

So, to be able to code it, we're going to use two resources:

- A tutorial made by Costa Huang. Costa is behind CleanRL, a Deep Reinforcement Learning library that provides high-quality single-file implementation with research-friendly features.
- In addition to the tutorial, to go deeper, you can read the 13 core implementation details: https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/

Then, to test its robustness, we're going to train it in:

- LunarLander-v2

Watch the original course video

And finally, we will push the trained model to the Hub to evaluate and visualize your agent playing.

LunarLander-v2 is the first environment you used when you started this course. At that time, you didn't know how it worked, and now you can code it from scratch and train it. **How incredible is that .**

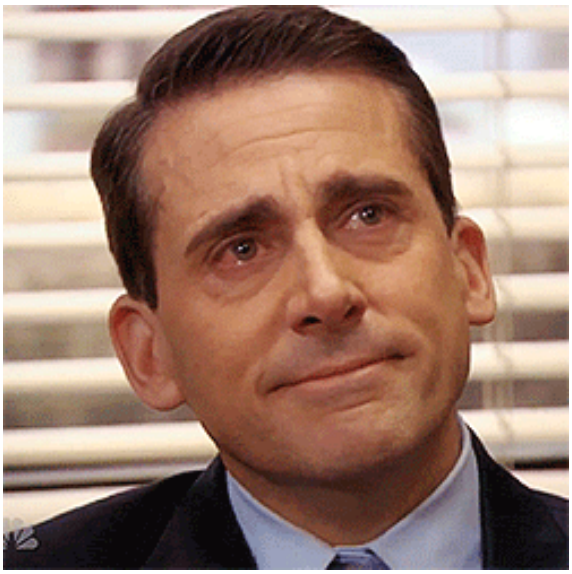


Figure 16: Reaction animation in the original course [original animation]

Let's get started!

The colab notebook:

Open original notebook in Colab

5.1 Unit 8: Proximal Policy Gradient (PPO) with PyTorch

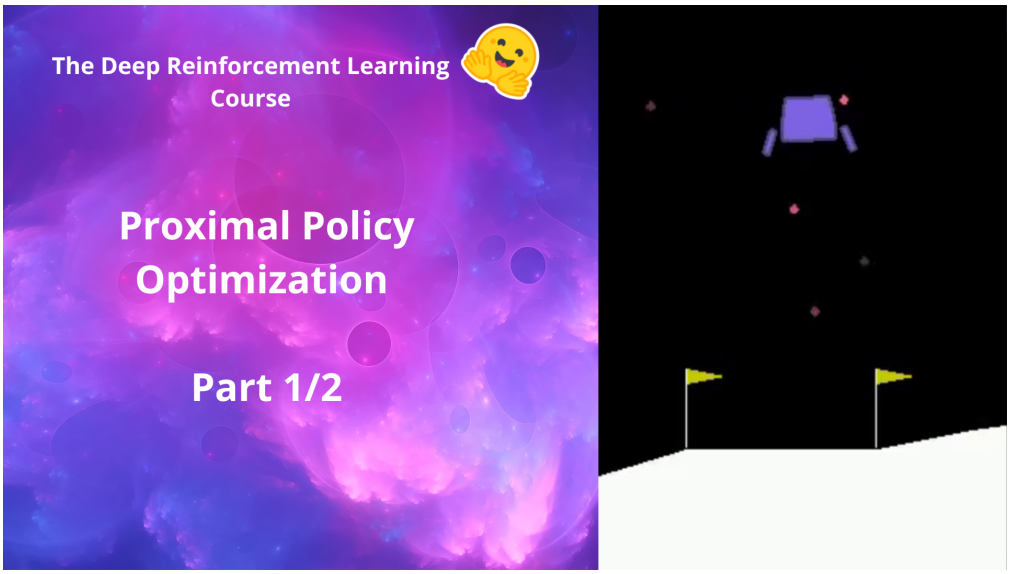


Figure 17: Hugging Face Deep RL Course, Unit 8: PPO [original figure]

In this notebook, you'll learn to **code your PPO agent from scratch with PyTorch using CleanRL implementation as model**.

To test its robustness, we're going to train it in:

- LunarLander-v2

We're constantly trying to improve our tutorials, so **if you find some issues in this notebook**, please open an issue on the GitHub Repo.

5.2 Objectives of this notebook

At the end of the notebook, you will:

- Be able to **code your PPO agent from scratch using PyTorch**.
- Be able to **push your trained agent and the code to the Hub** with a nice video replay and an evaluation score .

5.3 Prerequisites

Before diving into the notebook, you need to:

[ ] Study PPO by reading Unit 8

To validate this hands-on for the certification process, you need to push one model, we don't ask for a minimal result but we **advise you to try different hyperparameters settings to get better results**.

If you don't find your model, **go to the bottom of the page and click on the refresh button**

For more information about the certification process, check this section https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

5.4 Set the GPU

- To **accelerate the agent's training, we'll use a GPU**. To do that, go to `Runtime > Change Runtime type`

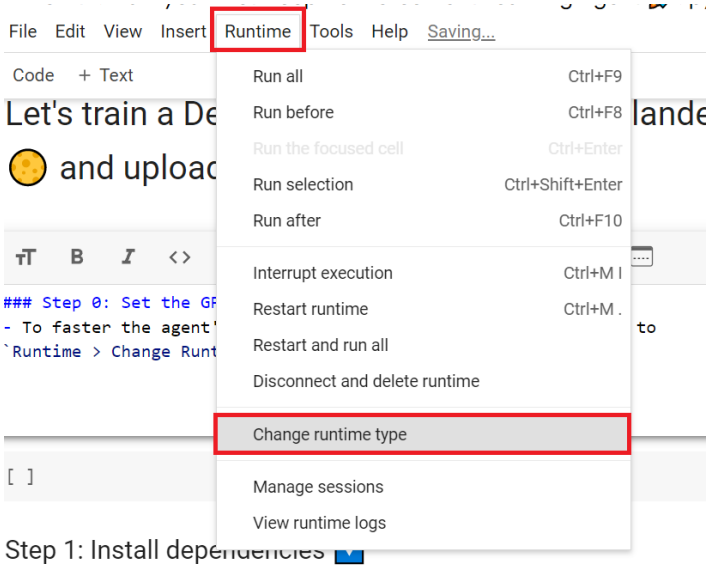


Figure 18: Google Colab menu for selecting the runtime [original figure]

- `Hardware Accelerator > GPU`

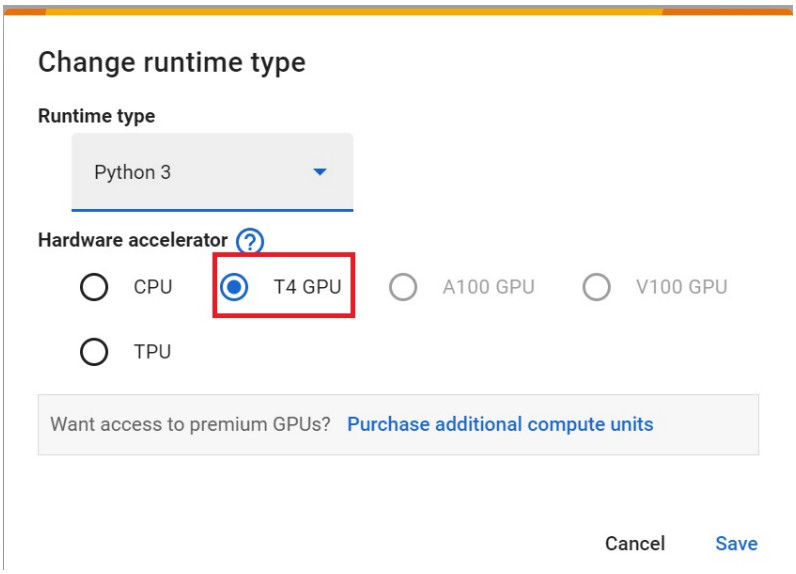


Figure 19: Google Colab GPU accelerator selection [original figure]

5.5 Create a virtual display

During the notebook, we'll need to generate a replay video. To do so, with colab, **we need to have a virtual screen to be able to render the environment** (and thus record the frames).

Hence the following cell will install the librairies and create and run a virtual screen

```
apt install python-opengl
apt install ffmpeg
apt install xvfb
pip install pyglet==1.5
pip install pyvirtualdisplay
```

```
# Virtual display
from pyvirtualdisplay import Display

virtual_display = Display(visible=0, size=(1400, 900))
virtual_display.start()
```

5.6 Install dependencies

For this exercise, we use `gym==0.21` because the video was recorded with Gym.

```
pip install gym==0.22
pip install imageio-ffmpeg
pip install huggingface_hub
pip install gym[box2d]==0.22
```

5.7 Let's code PPO from scratch with Costa Huang's tutorial

- For the core implementation of PPO we're going to use the excellent Costa Huang tutorial.
- In addition to the tutorial, to go deeper you can read the 37 core implementation details: https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/

The video tutorial: https://youtu.be/MEt6rrxH8W4

```
from IPython.display import HTML
```

```
HTML(
    '<iframe width="560" height="315" src="https://www.youtube.com/embed/MEt6rrxH8W4" title="
    YouTube video player" frameborder="0" allow="accelerometer; autoplay; clipboard-write;
    encrypted-media; gyroscope; picture-in-picture" allowfullscreen></iframe>'
)
```

5.8 Add the Hugging Face Integration

- In order to push our model to the Hub, we need to define a function `package_to_hub`
- Add dependencies we need to push our model to the Hub

```
from huggingface_hub import HfApi, upload_folder
from huggingface_hub.repocard import metadata_eval_result, metadata_save

from pathlib import Path
import datetime
import tempfile
import json
import shutil
import imageio

from wasabi import Printer

msg = Printer()
```

- Add new argument in `parse_args()` function to define the repo-id where we want to push the model.

```
# Adding HuggingFace argument
parser.add_argument(
    "--repo-id",
    type=str,
    default="ThomasSimonini/ppo-CartPole-v1",
    help="id of the model repository from the Hugging Face Hub {username/repo_name}",
)
```

- Next, we add the methods needed to push the model to the Hub
- These methods will:
  - `_evalutate_agent()`: evaluate the agent.
  - `_generate_model_card()`: generate the model card of your agent.
  - `_record_video()`: record a video of your agent.

```
def package_to_hub(
    repo_id,
    model,
    hyperparameters,
    eval_env,
    video_fps=30,
    commit_message="Push agent to the Hub",
    token=None,
    logs=None,
):
    """
    Evaluate, Generate a video and Upload a model to Hugging Face Hub.
    This method does the complete pipeline:
```

```
    - It evaluates the model
    - It generates the model card
    - It generates a replay video of the agent
    - It pushes everything to the hub
    :param repo_id: id of the model repository from the Hugging Face Hub
    :param model: trained model
    :param eval_env: environment used to evaluate the agent
    :param fps: number of fps for rendering the video
    :param commit_message: commit message
    :param logs: directory on local machine of tensorboard logs you'd like to upload
    """
    msg.info(
        "This function will save, evaluate, generate a video of your agent, "
        "create a model card and push everything to the hub. "
        "It might take up to 1min. \n "
        "This is a work in progress: if you encounter a bug, please open an issue."
    )
    # Step 1: Clone or create the repo
    repo_url = HfApi().create_repo(
        repo_id=repo_id,
        token=token,
        private=False,
        exist_ok=True,
    )

    with tempfile.TemporaryDirectory() as tmpdirname:
        tmpdirname = Path(tmpdirname)

        # Step 2: Save the model
        torch.save(model.state_dict(), tmpdirname / "model.pt")

        # Step 3: Evaluate the model and build JSON
        mean_reward, std_reward = _evaluate_agent(eval_env, 10, model)

        # First get datetime
        eval_datetime = datetime.datetime.now()
        eval_form_datetime = eval_datetime.isoformat()

        evaluate_data = {
            "env_id": hyperparameters.env_id,
            "mean_reward": mean_reward,
            "std_reward": std_reward,
            "n_evaluation_episodes": 10,
            "eval_datetime": eval_form_datetime,
        }

        # Write a JSON file
        with open(tmpdirname / "results.json", "w") as outfile:
            json.dump(evaluate_data, outfile)

        # Step 4: Generate a video
        video_path = tmpdirname / "replay.mp4"
        record_video(eval_env, model, video_path, video_fps)

        # Step 5: Generate the model card
        generated_model_card, metadata = _generate_model_card(
            "PPO", hyperparameters.env_id, mean_reward, std_reward, hyperparameters
        )
        _save_model_card(tmpdirname, generated_model_card, metadata)

        # Step 6: Add logs if needed
        if logs:
            _add_logdir(tmpdirname, Path(logs))

        msg.info(f"Pushing repo {repo_id} to the Hugging Face Hub")
```

```
        repo_url = upload_folder(
            repo_id=repo_id,
            folder_path=tmpdirname,
            path_in_repo="",
            commit_message=commit_message,
            token=token,
        )

        msg.info(f"Your model is pushed to the Hub. You can view your model here: {repo_url}")
    return repo_url


def _evaluate_agent(env, n_eval_episodes, policy):
    """
    Evaluate the agent for ``n_eval_episodes`` episodes and returns average reward and std of
    reward.
    :param env: The evaluation environment
    :param n_eval_episodes: Number of episode to evaluate the agent
    :param policy: The agent
    """
    episode_rewards = []
    for episode in range(n_eval_episodes):
        state = env.reset()
        step = 0
        done = False
        total_rewards_ep = 0

        while done is False:
            state = torch.Tensor(state).to(device)
            action, _, _, _ = policy.get_action_and_value(state)
            new_state, reward, done, info = env.step(action.cpu().numpy())
            total_rewards_ep += reward
            if done:
                break
            state = new_state
        episode_rewards.append(total_rewards_ep)
    mean_reward = np.mean(episode_rewards)
    std_reward = np.std(episode_rewards)

    return mean_reward, std_reward


def record_video(env, policy, out_directory, fps=30):
    images = []
    done = False
    state = env.reset()
    img = env.render(mode="rgb_array")
    images.append(img)
    while not done:
        state = torch.Tensor(state).to(device)
        # Take the action (index) that have the maximum expected future reward given that state
        action, _, _, _ = policy.get_action_and_value(state)
        state, reward, done, info = env.step(
            action.cpu().numpy()
        )  # We directly put next_state = state for recording logic
        img = env.render(mode="rgb_array")
        images.append(img)
    imageio.mimsave(out_directory, [np.array(img) for i, img in enumerate(images)], fps=fps)


def _generate_model_card(model_name, env_id, mean_reward, std_reward, hyperparameters):
    """
    Generate the model card for the Hub
    :param model_name: name of the model
```

```
    :env_id: name of the environment
    :mean_reward: mean reward of the agent
    :std_reward: standard deviation of the mean reward of the agent
    :hyperparameters: training arguments
    """
    # Step 1: Select the tags
    metadata = generate_metadata(model_name, env_id, mean_reward, std_reward)

    # Transform the hyperparams namespace to string
    converted_dict = vars(hyperparameters)
    converted_str = str(converted_dict)
    converted_str = converted_str.split(", ")
    converted_str = "\n".join(converted_str)

    # Step 2: Generate the model card
    model_card = f"""
  # PPO Agent Playing {env_id}

  This is a trained model of a PPO agent playing {env_id}.

  # Hyperparameters
  """
    return model_card, metadata


def generate_metadata(model_name, env_id, mean_reward, std_reward):
    """
    Define the tags for the model card
    :param model_name: name of the model
    :param env_id: name of the environment
    :mean_reward: mean reward of the agent
    :std_reward: standard deviation of the mean reward of the agent
    """
    metadata = {}
    metadata["tags"] = [
        env_id,
        "ppo",
        "deep-reinforcement-learning",
        "reinforcement-learning",
        "custom-implementation",
        "deep-rl-course",
    ]

    # Add metrics
    eval = metadata_eval_result(
        model_pretty_name=model_name,
        task_pretty_name="reinforcement-learning",
        task_id="reinforcement-learning",
        metrics_pretty_name="mean_reward",
        metrics_id="mean_reward",
        metrics_value=f"{mean_reward:.2f} +/- {std_reward:.2f}",
        dataset_pretty_name=env_id,
        dataset_id=env_id,
    )

    # Merges both dictionaries
    metadata = {**metadata, **eval}

    return metadata


def _save_model_card(local_path, generated_model_card, metadata):
    """Saves a model card for the repository.
    :param local_path: repository directory
    :param generated_model_card: model card generated by _generate_model_card()
```

```
    :param metadata: metadata
    """
    readme_path = local_path / "README.md"
    readme = ""
    if readme_path.exists():
        with readme_path.open("r", encoding="utf8") as f:
            readme = f.read()
    else:
        readme = generated_model_card

    with readme_path.open("w", encoding="utf-8") as f:
        f.write(readme)

    # Save our metrics to Readme metadata
    metadata_save(readme_path, metadata)


def _add_logdir(local_path: Path, logdir: Path):
    """Adds a logdir to the repository.
    :param local_path: repository directory
    :param logdir: logdir directory
    """
    if logdir.exists() and logdir.is_dir():
        # Add the logdir to the repository under new dir called logs
        repo_logdir = local_path / "logs"

        # Delete current logs if they exist
        if repo_logdir.exists():
            shutil.rmtree(repo_logdir)

        # Copy logdir into repo logdir
        shutil.copytree(logdir, repo_logdir)
```

- Finally, we call this function at the end of the PPO training

```
# Create the evaluation environment
eval_env = gym.make(args.env_id)

package_to_hub(
    repo_id=args.repo_id,
    model=agent,  # The model we want to save
    hyperparameters=args,
    eval_env=gym.make(args.env_id),
    logs=f"runs/{run_name}",
)
```

- Here's what the final ppo.py file looks like:

```
# docs and experiment results can be found at https://docs.cleanrl.dev/rl-algorithms/ppo/#ppopy

import argparse
import os
import random
import time
from distutils.util import strtobool

import gym
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
```

```
from torch.distributions.categorical import Categorical
from torch.utils.tensorboard import SummaryWriter

from huggingface_hub import HfApi, upload_folder
from huggingface_hub.repocard import metadata_eval_result, metadata_save

from pathlib import Path
import datetime
import tempfile
import json
import shutil
import imageio

from wasabi import Printer

msg = Printer()


def parse_args():
    # fmt: off
    parser = argparse.ArgumentParser()
    parser.add_argument("--exp-name", type=str, default=os.path.basename(__file__).rstrip(".py"),
        help="the name of this experiment")
    parser.add_argument("--seed", type=int, default=1,
        help="seed of the experiment")
    parser.add_argument("--torch-deterministic", type=lambda x: bool(strtobool(x)), default=True,
    nargs="?", const=True,
        help="if toggled, `torch.backends.cudnn.deterministic=False`")
    parser.add_argument("--cuda", type=lambda x: bool(strtobool(x)), default=True, nargs="?", const
    =True,
        help="if toggled, cuda will be enabled by default")
    parser.add_argument("--track", type=lambda x: bool(strtobool(x)), default=False, nargs="?",
    const=True,
        help="if toggled, this experiment will be tracked with Weights and Biases")
    parser.add_argument("--wandb-project-name", type=str, default="cleanRL",
        help="the wandb's project name")
    parser.add_argument("--wandb-entity", type=str, default=None,
        help="the entity (team) of wandb's project")
    parser.add_argument("--capture-video", type=lambda x: bool(strtobool(x)), default=False, nargs
    ="?", const=True,
        help="weather to capture videos of the agent performances (check out `videos` folder)")

    # Algorithm specific arguments
    parser.add_argument("--env-id", type=str, default="CartPole-v1",
        help="the id of the environment")
    parser.add_argument("--total-timesteps", type=int, default=50000,
        help="total timesteps of the experiments")
    parser.add_argument("--learning-rate", type=float, default=2.5e-4,
        help="the learning rate of the optimizer")
    parser.add_argument("--num-envs", type=int, default=4,
        help="the number of parallel game environments")
    parser.add_argument("--num-steps", type=int, default=128,
        help="the number of steps to run in each environment per policy rollout")
    parser.add_argument("--anneal-lr", type=lambda x: bool(strtobool(x)), default=True, nargs="?",
    const=True,
        help="Toggle learning rate annealing for policy and value networks")
    parser.add_argument("--gae", type=lambda x: bool(strtobool(x)), default=True, nargs="?", const=
    True,
        help="Use GAE for advantage computation")
    parser.add_argument("--gamma", type=float, default=0.99,
        help="the discount factor gamma")
    parser.add_argument("--gae-lambda", type=float, default=0.95,
        help="the lambda for the general advantage estimation")
    parser.add_argument("--num-minibatches", type=int, default=4,
        help="the number of mini-batches")
```

```
    parser.add_argument("--update-epochs", type=int, default=4,
        help="the K epochs to update the policy")
    parser.add_argument("--norm-adv", type=lambda x: bool(strtobool(x)), default=True, nargs="?",
    const=True,
        help="Toggles advantages normalization")
    parser.add_argument("--clip-coef", type=float, default=0.2,
        help="the surrogate clipping coefficient")
    parser.add_argument("--clip-vloss", type=lambda x: bool(strtobool(x)), default=True, nargs="?",
     const=True,
        help="Toggles whether or not to use a clipped loss for the value function, as per the paper
    .")
    parser.add_argument("--ent-coef", type=float, default=0.01,
        help="coefficient of the entropy")
    parser.add_argument("--vf-coef", type=float, default=0.5,
        help="coefficient of the value function")
    parser.add_argument("--max-grad-norm", type=float, default=0.5,
        help="the maximum norm for the gradient clipping")
    parser.add_argument("--target-kl", type=float, default=None,
        help="the target KL divergence threshold")

    # Adding HuggingFace argument
    parser.add_argument("--repo-id", type=str, default="ThomasSimonini/ppo-CartPole-v1", help="id
    of the model repository from the Hugging Face Hub {username/repo_name}")

    args = parser.parse_args()
    args.batch_size = int(args.num_envs * args.num_steps)
    args.minibatch_size = int(args.batch_size // args.num_minibatches)
    # fmt: on
    return args


def package_to_hub(
    repo_id,
    model,
    hyperparameters,
    eval_env,
    video_fps=30,
    commit_message="Push agent to the Hub",
    token=None,
    logs=None,
):
    """
    Evaluate, Generate a video and Upload a model to Hugging Face Hub.
    This method does the complete pipeline:
    - It evaluates the model
    - It generates the model card
    - It generates a replay video of the agent
    - It pushes everything to the hub
    :param repo_id: id of the model repository from the Hugging Face Hub
    :param model: trained model
    :param eval_env: environment used to evaluate the agent
    :param fps: number of fps for rendering the video
    :param commit_message: commit message
    :param logs: directory on local machine of tensorboard logs you'd like to upload
    """
    msg.info(
        "This function will save, evaluate, generate a video of your agent, "
        "create a model card and push everything to the hub. "
        "It might take up to 1min. \n "
        "This is a work in progress: if you encounter a bug, please open an issue."
    )
    # Step 1: Clone or create the repo
    repo_url = HfApi().create_repo(
        repo_id=repo_id,
        token=token,
```

```
        private=False,
        exist_ok=True,
    )

    with tempfile.TemporaryDirectory() as tmpdirname:
        tmpdirname = Path(tmpdirname)

        # Step 2: Save the model
        torch.save(model.state_dict(), tmpdirname / "model.pt")

        # Step 3: Evaluate the model and build JSON
        mean_reward, std_reward = _evaluate_agent(eval_env, 10, model)

        # First get datetime
        eval_datetime = datetime.datetime.now()
        eval_form_datetime = eval_datetime.isoformat()

        evaluate_data = {
            "env_id": hyperparameters.env_id,
            "mean_reward": mean_reward,
            "std_reward": std_reward,
            "n_evaluation_episodes": 10,
            "eval_datetime": eval_form_datetime,
        }

        # Write a JSON file
        with open(tmpdirname / "results.json", "w") as outfile:
            json.dump(evaluate_data, outfile)

        # Step 4: Generate a video
        video_path = tmpdirname / "replay.mp4"
        record_video(eval_env, model, video_path, video_fps)

        # Step 5: Generate the model card
        generated_model_card, metadata = _generate_model_card(
            "PPO", hyperparameters.env_id, mean_reward, std_reward, hyperparameters
        )
        _save_model_card(tmpdirname, generated_model_card, metadata)

        # Step 6: Add logs if needed
        if logs:
            _add_logdir(tmpdirname, Path(logs))

        msg.info(f"Pushing repo {repo_id} to the Hugging Face Hub")

        repo_url = upload_folder(
            repo_id=repo_id,
            folder_path=tmpdirname,
            path_in_repo="",
            commit_message=commit_message,
            token=token,
        )

        msg.info(f"Your model is pushed to the Hub. You can view your model here: {repo_url}")
    return repo_url


def _evaluate_agent(env, n_eval_episodes, policy):
    """
    Evaluate the agent for ``n_eval_episodes`` episodes and returns average reward and std of
    reward.
    :param env: The evaluation environment
    :param n_eval_episodes: Number of episode to evaluate the agent
    :param policy: The agent
    """
```

```
    episode_rewards = []
    for episode in range(n_eval_episodes):
        state = env.reset()
        step = 0
        done = False
        total_rewards_ep = 0

        while done is False:
            state = torch.Tensor(state).to(device)
            action, _, _, _ = policy.get_action_and_value(state)
            new_state, reward, done, info = env.step(action.cpu().numpy())
            total_rewards_ep += reward
            if done:
                break
            state = new_state
        episode_rewards.append(total_rewards_ep)
    mean_reward = np.mean(episode_rewards)
    std_reward = np.std(episode_rewards)

    return mean_reward, std_reward


def record_video(env, policy, out_directory, fps=30):
    images = []
    done = False
    state = env.reset()
    img = env.render(mode="rgb_array")
    images.append(img)
    while not done:
        state = torch.Tensor(state).to(device)
        # Take the action (index) that have the maximum expected future reward given that state
        action, _, _, _ = policy.get_action_and_value(state)
        state, reward, done, info = env.step(
            action.cpu().numpy()
        )  # We directly put next_state = state for recording logic
        img = env.render(mode="rgb_array")
        images.append(img)
    imageio.mimsave(out_directory, [np.array(img) for i, img in enumerate(images)], fps=fps)


def _generate_model_card(model_name, env_id, mean_reward, std_reward, hyperparameters):
    """
    Generate the model card for the Hub
    :param model_name: name of the model
    :env_id: name of the environment
    :mean_reward: mean reward of the agent
    :std_reward: standard deviation of the mean reward of the agent
    :hyperparameters: training arguments
    """
    # Step 1: Select the tags
    metadata = generate_metadata(model_name, env_id, mean_reward, std_reward)

    # Transform the hyperparams namespace to string
    converted_dict = vars(hyperparameters)
    converted_str = str(converted_dict)
    converted_str = converted_str.split(", ")
    converted_str = "\n".join(converted_str)

    # Step 2: Generate the model card
    model_card = f"""
  # PPO Agent Playing {env_id}

  This is a trained model of a PPO agent playing {env_id}.

  # Hyperparameters
```

```
  """
    return model_card, metadata


def generate_metadata(model_name, env_id, mean_reward, std_reward):
    """
    Define the tags for the model card
    :param model_name: name of the model
    :param env_id: name of the environment
    :mean_reward: mean reward of the agent
    :std_reward: standard deviation of the mean reward of the agent
    """
    metadata = {}
    metadata["tags"] = [
        env_id,
        "ppo",
        "deep-reinforcement-learning",
        "reinforcement-learning",
        "custom-implementation",
        "deep-rl-course",
    ]

    # Add metrics
    eval = metadata_eval_result(
        model_pretty_name=model_name,
        task_pretty_name="reinforcement-learning",
        task_id="reinforcement-learning",
        metrics_pretty_name="mean_reward",
        metrics_id="mean_reward",
        metrics_value=f"{mean_reward:.2f} +/- {std_reward:.2f}",
        dataset_pretty_name=env_id,
        dataset_id=env_id,
    )

    # Merges both dictionaries
    metadata = {**metadata, **eval}

    return metadata


def _save_model_card(local_path, generated_model_card, metadata):
    """Saves a model card for the repository.
    :param local_path: repository directory
    :param generated_model_card: model card generated by _generate_model_card()
    :param metadata: metadata
    """
    readme_path = local_path / "README.md"
    readme = ""
    if readme_path.exists():
        with readme_path.open("r", encoding="utf8") as f:
            readme = f.read()
    else:
        readme = generated_model_card

    with readme_path.open("w", encoding="utf-8") as f:
        f.write(readme)

    # Save our metrics to Readme metadata
    metadata_save(readme_path, metadata)


def _add_logdir(local_path: Path, logdir: Path):
    """Adds a logdir to the repository.
    :param local_path: repository directory
    :param logdir: logdir directory
```

```
    """
    if logdir.exists() and logdir.is_dir():
        # Add the logdir to the repository under new dir called logs
        repo_logdir = local_path / "logs"

        # Delete current logs if they exist
        if repo_logdir.exists():
            shutil.rmtree(repo_logdir)

        # Copy logdir into repo logdir
        shutil.copytree(logdir, repo_logdir)


def make_env(env_id, seed, idx, capture_video, run_name):
    def thunk():
        env = gym.make(env_id)
        env = gym.wrappers.RecordEpisodeStatistics(env)
        if capture_video:
            if idx == 0:
                env = gym.wrappers.RecordVideo(env, f"videos/{run_name}")
        env.seed(seed)
        env.action_space.seed(seed)
        env.observation_space.seed(seed)
        return env

    return thunk


def layer_init(layer, std=np.sqrt(2), bias_const=0.0):
    torch.nn.init.orthogonal_(layer.weight, std)
    torch.nn.init.constant_(layer.bias, bias_const)
    return layer


class Agent(nn.Module):
    def __init__(self, envs):
        super().__init__()
        self.critic = nn.Sequential(
            layer_init(nn.Linear(np.array(envs.single_observation_space.shape).prod(), 64)),
            nn.Tanh(),
            layer_init(nn.Linear(64, 64)),
            nn.Tanh(),
            layer_init(nn.Linear(64, 1), std=1.0),
        )
        self.actor = nn.Sequential(
            layer_init(nn.Linear(np.array(envs.single_observation_space.shape).prod(), 64)),
            nn.Tanh(),
            layer_init(nn.Linear(64, 64)),
            nn.Tanh(),
            layer_init(nn.Linear(64, envs.single_action_space.n), std=0.01),
        )

    def get_value(self, x):
        return self.critic(x)

    def get_action_and_value(self, x, action=None):
        logits = self.actor(x)
        probs = Categorical(logits=logits)
        if action is None:
            action = probs.sample()
        return action, probs.log_prob(action), probs.entropy(), self.critic(x)


if __name__ == "__main__":
    args = parse_args()
```

```
    run_name = f"{args.env_id}__{args.exp_name}__{args.seed}__{int(time.time())}"
    if args.track:
        import wandb

        wandb.init(
            project=args.wandb_project_name,
            entity=args.wandb_entity,
            sync_tensorboard=True,
            config=vars(args),
            name=run_name,
            monitor_gym=True,
            save_code=True,
        )
    writer = SummaryWriter(f"runs/{run_name}")
    writer.add_text(
        "hyperparameters",
        "|param|value|\n|-|-|\n%s" % ("\n".join([f"|{key}|{value}|" for key, value in vars(args).
    items()])),
    )

    # TRY NOT TO MODIFY: seeding
    random.seed(args.seed)
    np.random.seed(args.seed)
    torch.manual_seed(args.seed)
    torch.backends.cudnn.deterministic = args.torch_deterministic

    device = torch.device("cuda" if torch.cuda.is_available() and args.cuda else "cpu")

    # env setup
    envs = gym.vector.SyncVectorEnv(
        [make_env(args.env_id, args.seed + i, i, args.capture_video, run_name) for i in range(args.
    num_envs)]
    )
    assert isinstance(envs.single_action_space, gym.spaces.Discrete), "only discrete action space
    is supported"

    agent = Agent(envs).to(device)
    optimizer = optim.Adam(agent.parameters(), lr=args.learning_rate, eps=1e-5)

    # ALGO Logic: Storage setup
    obs = torch.zeros((args.num_steps, args.num_envs) + envs.single_observation_space.shape).to(
    device)
    actions = torch.zeros((args.num_steps, args.num_envs) + envs.single_action_space.shape).to(
    device)
    logprobs = torch.zeros((args.num_steps, args.num_envs)).to(device)
    rewards = torch.zeros((args.num_steps, args.num_envs)).to(device)
    dones = torch.zeros((args.num_steps, args.num_envs)).to(device)
    values = torch.zeros((args.num_steps, args.num_envs)).to(device)

    # TRY NOT TO MODIFY: start the game
    global_step = 0
    start_time = time.time()
    next_obs = torch.Tensor(envs.reset()).to(device)
    next_done = torch.zeros(args.num_envs).to(device)
    num_updates = args.total_timesteps // args.batch_size

    for update in range(1, num_updates + 1):
        # Annealing the rate if instructed to do so.
        if args.anneal_lr:
            frac = 1.0 - (update - 1.0) / num_updates
            lrnow = frac * args.learning_rate
            optimizer.param_groups[0]["lr"] = lrnow

        for step in range(0, args.num_steps):
            global_step += 1 * args.num_envs
```

```
            obs[step] = next_obs
            dones[step] = next_done

            # ALGO LOGIC: action logic
            with torch.no_grad():
                action, logprob, _, value = agent.get_action_and_value(next_obs)
                values[step] = value.flatten()
            actions[step] = action
            logprobs[step] = logprob

            # TRY NOT TO MODIFY: execute the game and log data.
            next_obs, reward, done, info = envs.step(action.cpu().numpy())
            rewards[step] = torch.tensor(reward).to(device).view(-1)
            next_obs, next_done = torch.Tensor(next_obs).to(device), torch.Tensor(done).to(device)

            for item in info:
                if "episode" in item.keys():
                    print(f"global_step={global_step}, episodic_return={item['episode']['r']}")
                    writer.add_scalar("charts/episodic_return", item["episode"]["r"], global_step)
                    writer.add_scalar("charts/episodic_length", item["episode"]["l"], global_step)
                    break

        # bootstrap value if not done
        with torch.no_grad():
            next_value = agent.get_value(next_obs).reshape(1, -1)
            if args.gae:
                advantages = torch.zeros_like(rewards).to(device)
                lastgaelam = 0
                for t in reversed(range(args.num_steps)):
                    if t == args.num_steps - 1:
                        nextnonterminal = 1.0 - next_done
                        nextvalues = next_value
                    else:
                        nextnonterminal = 1.0 - dones[t + 1]
                        nextvalues = values[t + 1]
                    delta = rewards[t] + args.gamma * nextvalues * nextnonterminal - values[t]
                    advantages[t] = lastgaelam = delta + args.gamma * args.gae_lambda *
    nextnonterminal * lastgaelam
                returns = advantages + values
            else:
                returns = torch.zeros_like(rewards).to(device)
                for t in reversed(range(args.num_steps)):
                    if t == args.num_steps - 1:
                        nextnonterminal = 1.0 - next_done
                        next_return = next_value
                    else:
                        nextnonterminal = 1.0 - dones[t + 1]
                        next_return = returns[t + 1]
                    returns[t] = rewards[t] + args.gamma * nextnonterminal * next_return
                advantages = returns - values

        # flatten the batch
        b_obs = obs.reshape((-1,) + envs.single_observation_space.shape)
        b_logprobs = logprobs.reshape(-1)
        b_actions = actions.reshape((-1,) + envs.single_action_space.shape)
        b_advantages = advantages.reshape(-1)
        b_returns = returns.reshape(-1)
        b_values = values.reshape(-1)

        # Optimizing the policy and value network
        b_inds = np.arange(args.batch_size)
        clipfracs = []
        for epoch in range(args.update_epochs):
            np.random.shuffle(b_inds)
            for start in range(0, args.batch_size, args.minibatch_size):
```

```
                end = start + args.minibatch_size
                mb_inds = b_inds[start:end]

                _, newlogprob, entropy, newvalue = agent.get_action_and_value(
                    b_obs[mb_inds], b_actions.long()[mb_inds]
                )
                logratio = newlogprob - b_logprobs[mb_inds]
                ratio = logratio.exp()

                with torch.no_grad():
                    # calculate approx_kl http://joschu.net/blog/kl-approx.html
                    old_approx_kl = (-logratio).mean()
                    approx_kl = ((ratio - 1) - logratio).mean()
                    clipfracs += [((ratio - 1.0).abs() > args.clip_coef).float().mean().item()]

                mb_advantages = b_advantages[mb_inds]
                if args.norm_adv:
                    mb_advantages = (mb_advantages - mb_advantages.mean()) / (mb_advantages.std() +
 1e-8)

                # Policy loss
                pg_loss1 = -mb_advantages * ratio
                pg_loss2 = -mb_advantages * torch.clamp(ratio, 1 - args.clip_coef, 1 + args.
clip_coef)
                pg_loss = torch.max(pg_loss1, pg_loss2).mean()

                # Value loss
                newvalue = newvalue.view(-1)
                if args.clip_vloss:
                    v_loss_unclipped = (newvalue - b_returns[mb_inds]) ** 2
                    v_clipped = b_values[mb_inds] + torch.clamp(
                        newvalue - b_values[mb_inds],
                        -args.clip_coef,
                        args.clip_coef,
                    )
                    v_loss_clipped = (v_clipped - b_returns[mb_inds]) ** 2
                    v_loss_max = torch.max(v_loss_unclipped, v_loss_clipped)
                    v_loss = 0.5 * v_loss_max.mean()
                else:
                    v_loss = 0.5 * ((newvalue - b_returns[mb_inds]) ** 2).mean()

                entropy_loss = entropy.mean()
                loss = pg_loss - args.ent_coef * entropy_loss + v_loss * args.vf_coef

                optimizer.zero_grad()
                loss.backward()
                nn.utils.clip_grad_norm_(agent.parameters(), args.max_grad_norm)
                optimizer.step()

            if args.target_kl is not None:
                if approx_kl > args.target_kl:
                    break

        y_pred, y_true = b_values.cpu().numpy(), b_returns.cpu().numpy()
        var_y = np.var(y_true)
        explained_var = np.nan if var_y == 0 else 1 - np.var(y_true - y_pred) / var_y

        # TRY NOT TO MODIFY: record rewards for plotting purposes
        writer.add_scalar("charts/learning_rate", optimizer.param_groups[0]["lr"], global_step)
        writer.add_scalar("losses/value_loss", v_loss.item(), global_step)
        writer.add_scalar("losses/policy_loss", pg_loss.item(), global_step)
        writer.add_scalar("losses/entropy", entropy_loss.item(), global_step)
        writer.add_scalar("losses/old_approx_kl", old_approx_kl.item(), global_step)
        writer.add_scalar("losses/approx_kl", approx_kl.item(), global_step)
        writer.add_scalar("losses/clipfrac", np.mean(clipfracs), global_step)
```

```
        writer.add_scalar("losses/explained_variance", explained_var, global_step)
        print("SPS:", int(global_step / (time.time() - start_time)))
        writer.add_scalar("charts/SPS", int(global_step / (time.time() - start_time)), global_step)

    envs.close()
    writer.close()

    # Create the evaluation environment
    eval_env = gym.make(args.env_id)

    package_to_hub(
        repo_id=args.repo_id,
        model=agent,  # The model we want to save
        hyperparameters=args,
        eval_env=gym.make(args.env_id),
        logs=f"runs/{run_name}",
    )
```

To be able to share your model with the community there are three more steps to follow:

1. (If it's not already done) create an account to HF -> https://huggingface.co/join
2. Sign in and get your authentication token from the Hugging Face website.

- Create a new token (https://huggingface.co/settings/tokens) **with write role**

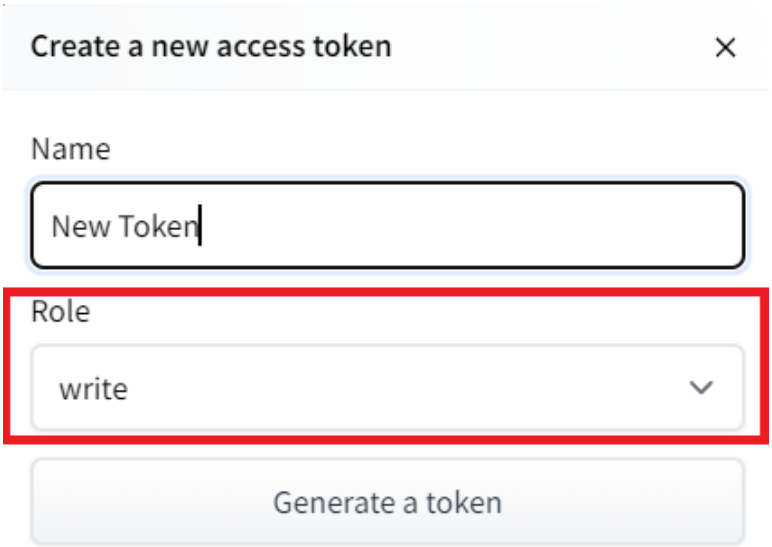


Figure 20: Creating a Hugging Face write-access token [original figure]

- Copy the token
- Run the cell below and paste the token

```
from huggingface_hub import notebook_login
notebook_login()
!git config --global credential.helper store
```

If you don't want to use Google Colab or a Jupyter Notebook, you need to use this command instead: `huggingface-cli login`

5.9 Let's start the training

Warning: Warning: Warning: Don't use **the same repo id with the one you used for the Unit 1**

- Now that you've coded PPO from scratch and added the Hugging Face Integration, we're ready to start the training
- First, you need to copy all your code to a file you create called `ppo.py`

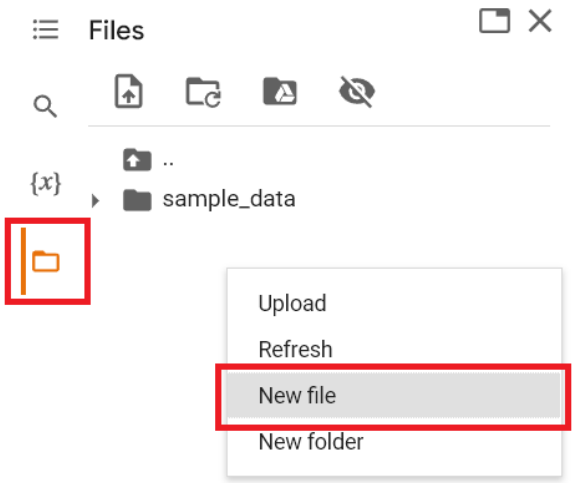


Figure 21: Create the Python file for PPO implementation [original figure]

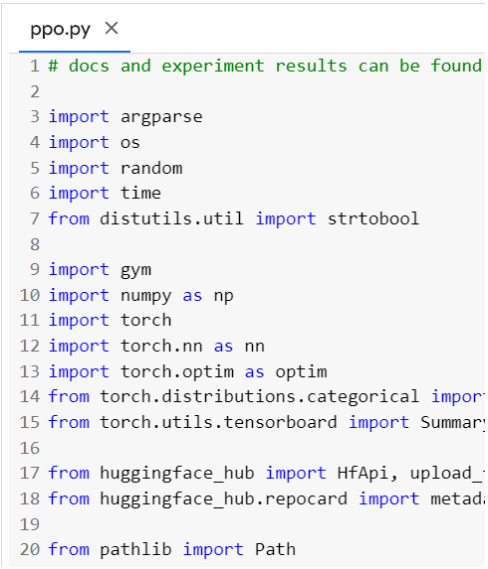


Figure 22: Prepare and save the PPO Python script [original figure]

- Now we just need to run this python script using `python <name-of-python-script>.py` with the additional parameters we defined using `argparse`
- You should modify more hyperparameters otherwise the training will not be super stable.

```
!python ppo.py --env-id="LunarLander-v2" --repo-id="YOUR_REPO_ID" --total-timesteps=50000
```

5.10 Some additional challenges

The best way to learn **is to try things on your own**! Why not try another environment? Or why not trying to modify the implementation to work with Gymnasium?

See you in Unit 8, part 2 where we're going to train agents to play Doom

5.11 Keep learning, stay awesome

6 Conclusion

That's all for today. Congrats on finishing this unit and the tutorial!

The best way to learn is to practice and try stuff. **Why not improve the implementation to handle frames as input?**.

See you on second part of this Unit

6.1 Keep Learning, Stay awesome

7 Additional Readings

These are **optional readings** if you want to go deeper.

7.1 PPO Explained

- Towards Delivering a Coherent Self-Contained Explanation of Proximal Policy Optimization by Daniel Bick
- What is the way to understand Proximal Policy Optimization Algorithm in RL?
- Foundations of Deep RL Series, L4 TRPO and PPO by Pieter Abbeel
- OpenAI PPO Blogpost
- Spinning Up RL PPO
- Paper Proximal Policy Optimization Algorithms

7.2 PPO Implementation details

- The 37 Implementation Details of Proximal Policy Optimization
- Part 1 of 3 — Proximal Policy Optimization Implementation: 11 Core Implementation Details

7.3 Importance Sampling

- Importance Sampling Explained

8 Introduction to PPO with Sample-Factory



Figure 23: Unit 8, Part 2: PPO with Sample Factory and ViZDoom [original figure]

In this second part of Unit 8, we'll get deeper into PPO optimization by using Sample-Factory, an **asynchronous implementation of the PPO algorithm**, to train our agent to play vizdoom (an open source version of Doom).

In the notebook, **you'll train your agent to play the Health Gathering level**, where the agent must collect health packs to avoid dying. After that, you can **train your agent to play more complex levels, such as Deathmatch**.



Figure 24: Environments shown in the course, including ViZDoom [original figure]

Sound exciting? Let's get started!

The hands-on is made by Edward Beeching, a Machine Learning Research Scientist at Hugging Face. He worked on Godot Reinforcement Learning Agents, an open-source interface for developing environments and agents in the Godot Game Engine.

9 Hands-on: advanced Deep Reinforcement Learning. Using Sample Factory to play Doom from pixels

Original Google Colab notebook

The colab notebook: Open original notebook in Colab

9.1 Unit 8 Part 2: Advanced Deep Reinforcement Learning. Using Sample Factory to play Doom from pixels



Figure 25: Unit 8, Part 2: PPO with Sample Factory and ViZDoom [original figure]

In this notebook, we will learn how to train a Deep Neural Network to collect objects in a 3D environment based on the game of Doom, a video of the resulting policy is shown below. We train this policy using Sample Factory, an asynchronous implementation of the PPO algorithm.

Please note the following points:

- Sample Factory is an advanced RL framework and **only functions on Linux and Mac** (not Windows).
- The framework performs best on a **GPU machine with many CPU cores**, where it can achieve speeds of 100k interactions per second. The resources available on a standard Colab notebook **limit the performance of this library**. So the speed in this setting **does not reflect the real-world performance**.
- Benchmarks for Sample Factory are available in a number of settings, check out the examples if you want to find out more.

```
from IPython.display import HTML

HTML(
    """<video width="640" height="480" controls>
  <source src="https://huggingface.co/edbeeching/doom_health_gathering_supreme_3333/resolve/main/
    replay.mp4"
  type="video/mp4">Your browser does not support the video tag.</video>"""
)
```

To validate this hands-on for the certification process, you need to push one model:

- `doom_health_gathering_supreme` get a result of >= 5.

To find your result, go to the leaderboard and find your model, **the result = mean_reward - std of reward**

If you don't find your model, **go to the bottom of the page and click on the refresh button**

For more information about the certification process, check this section https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

9.2 Set the GPU

- To **accelerate the agent's training, we'll use a GPU**. To do that, go to `Runtime > Change Runtime type`

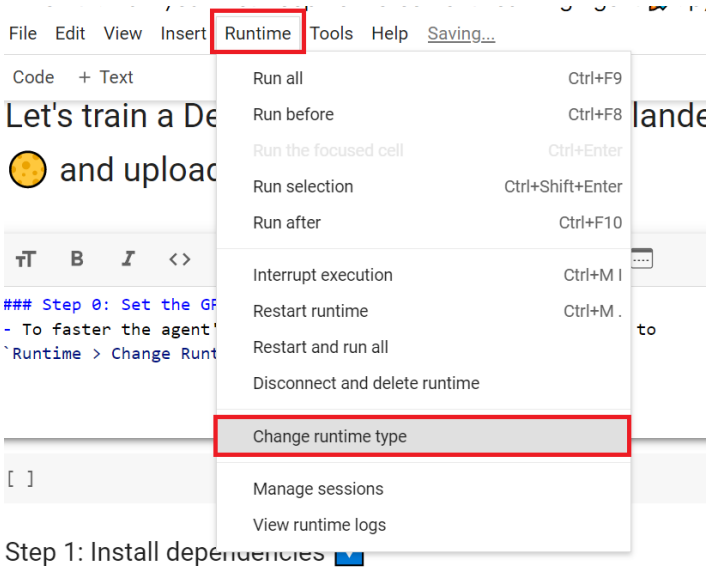


Figure 26: Google Colab menu for selecting the runtime [original figure]

- `Hardware Accelerator > GPU`

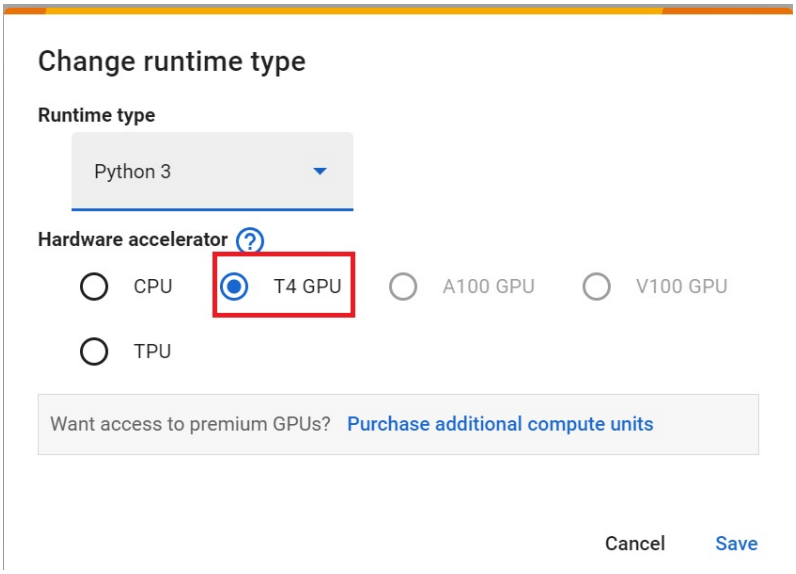


Figure 27: Google Colab GPU accelerator selection [original figure]

Before starting to train our agent, let's **study the library and environments we're going to use**.

9.3 Sample Factory

Sample Factory is one of the **fastest RL libraries focused on very efficient synchronous and asynchronous implementations of policy gradients (PPO)**.

Sample Factory is thoroughly **tested, used by many researchers and practitioners**, and is actively maintained. Our implementation is known to **reach SOTA performance in a variety of domains while minimizing RL experiment training time and hardware requirements**.

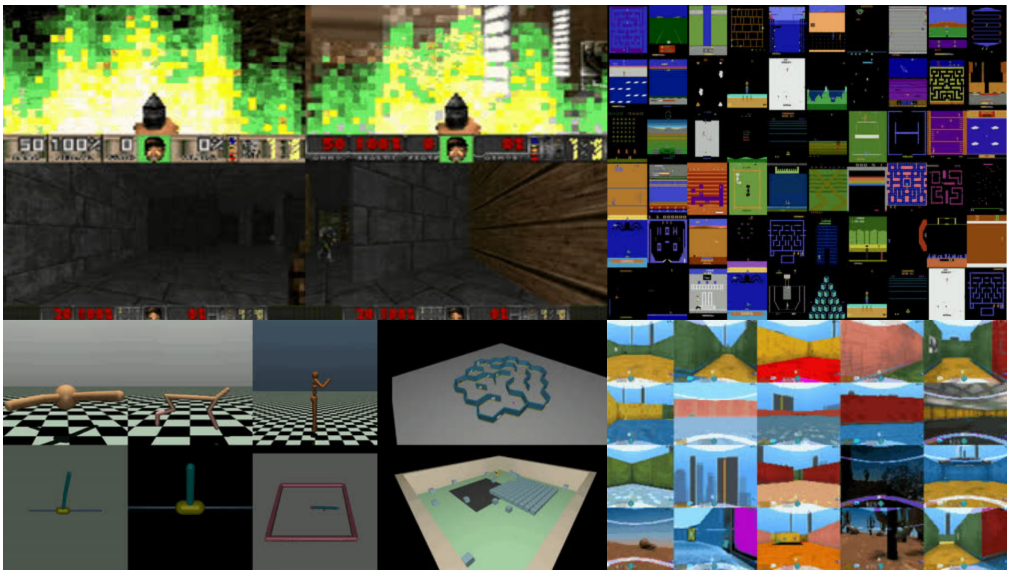


Figure 28: Environments supported by Sample Factory [original figure]

Key features

- Highly optimized algorithm architecture for maximum learning throughput
- Synchronous and asynchronous training regimes
- Serial (single-process) mode for easy debugging
- Optimal performance in both CPU-based and GPU-accelerated environments
- Single- & multi-agent training, self-play, supports training multiple policies at once on one or many GPUs
- Population-Based Training (PBT)
- Discrete, continuous, hybrid action spaces
- Vector-based, image-based, dictionary observation spaces
- Automatically creates a model architecture by parsing action/observation space specification. Supports custom model architectures
- Designed to be imported into other projects, custom environments are first-class citizens
- Detailed WandB and Tensorboard summaries, custom metrics
- HuggingFace integration (upload trained models and metrics to the Hub)
- Multiple example environment integrations with tuned parameters and trained models

All of the above policies are available on the hub. Search for the tag sample-factory

How sample-factory works

Sample-factory is one of the **most highly optimized RL implementations available to the community**.

It works by **spawning multiple processes that run rollout workers, inference workers and a learner worker**.

The *workers* **communicate through shared memory, which lowers the communication cost between processes**.

The *rollout workers* interact with the environment and send observations to the *inference workers*.

The *inferences workers* query a fixed version of the policy and **send actions back to the rollout worker**.

After k steps the rollout works send a trajectory of experience to the learner worker, **which it uses to update the agent's policy network**.

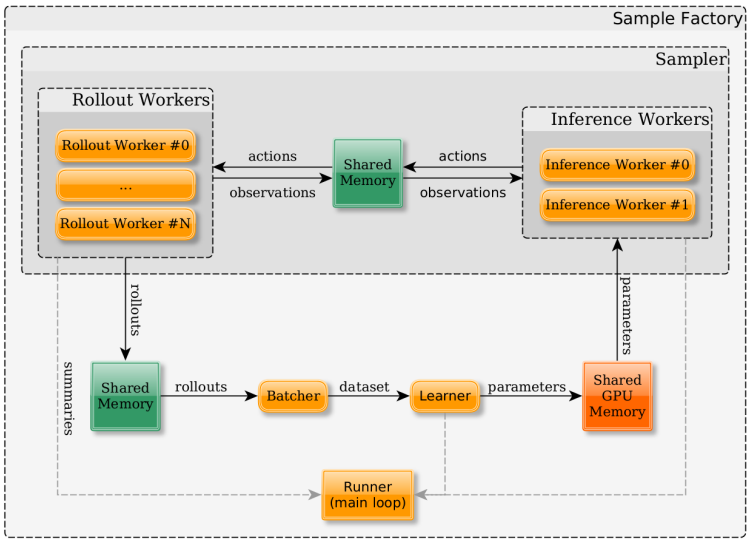


Figure 29: Sample Factory distributed rollout, inference and learner architecture [original figure]

Actor Critic models in Sample-factory

Actor Critic models in Sample Factory are composed of three components:

- **Encoder** - Process input observations (images, vectors) and map them to a vector. This is the part of the model you will most likely want to customize.
- **Core** - Intergrate vectors from one or more encoders, can optionally include a single- or multi-layer LSTM/GRU in a memory-based agent.
- **Decoder** - Apply additional layers to the output of the model core before computing the policy and value outputs.

The library has been designed to automatically support any observation and action spaces. Users can easily add their custom models. You can find out more in the documentation.

9.4 ViZDoom

ViZDoom is an **open-source python interface for the Doom Engine**.

The library was created in 2016 by Marek Wydmuch, Michal Kempka at the Institute of Computing Science, Poznan University of Technology, Poland.

The library enables the **training of agents directly from the screen pixels in a number of scenarios**, including team deathmatch, shown in the video below. Because the ViZDoom environment is based on a game the was created in the 90s, it can be run on modern hardware at accelerated speeds, **allowing us to learn complex AI behaviors fairly quickly**.

The library includes feature such as:

- Multi-platform (Linux, macOS, Windows),
- API for Python and C++,
- OpenAI Gym environment wrappers
- Easy-to-create custom scenarios (visual editors, scripting language, and examples available),
- Async and sync single-player and multiplayer modes,
- Lightweight (few MBs) and fast (up to 7000 fps in sync mode, single-threaded),
- Customizable resolution and rendering parameters,
- Access to the depth buffer (3D vision),
- Automatic labeling of game objects visible in the frame,
- Access to the audio buffer
- Access to the list of actors/objects and map geometry,

- Off-screen rendering and episode recording,
- Time scaling in async mode.

9.5 We first need to install some dependencies that are required for the ViZDoom environment

Now that our Colab runtime is set up, we can start by installing the dependencies required to run ViZDoom on linux.

If you are following on your machine on Mac, you will want to follow the installation instructions on the github page.

```
# Install ViZDoom deps from
# https://github.com/mwydmuch/ViZDoom/blob/master/doc/Building.md#-linux

apt-get install build-essential zlib1g-dev libsdl2-dev libjpeg-dev \
nasm tar libbz2-dev libgtk2.0-dev cmake git libfluidsynth-dev libgme-dev \
libopenal-dev timidity libwildmidi-dev unzip ffmpeg

# Boost libraries
apt-get install libboost-all-dev

# Lua binding dependencies
apt-get install liblua5.1-dev
```

9.6 Then we can install Sample Factory and ViZDoom

- This can take 7min

```
pip install sample-factory
pip install vizdoom
```

9.7 Setting up the Doom Environment in sample-factory

```
import functools

from sample_factory.algo.utils.context import global_model_factory
from sample_factory.cfg.arguments import parse_full_cfg, parse_sf_args
from sample_factory.envs.env_utils import register_env
from sample_factory.train import run_rl

from sf_examples.vizdoom.doom.doom_model import make_vizdoom_encoder
from sf_examples.vizdoom.doom.doom_params import add_doom_env_args, doom_override_defaults
from sf_examples.vizdoom.doom.doom_utils import DOOM_ENVS, make_doom_env_from_spec


# Registers all the ViZDoom environments
def register_vizdoom_envs():
    for env_spec in DOOM_ENVS:
        make_env_func = functools.partial(make_doom_env_from_spec, env_spec)
        register_env(env_spec.name, make_env_func)


# Sample Factory allows the registration of a custom Neural Network architecture
# See https://github.com/alex-petrenko/sample-factory/blob/master/sf_examples/vizdoom/doom/
    doom_model.py for more details
def register_vizdoom_models():
    global_model_factory().register_encoder_factory(make_vizdoom_encoder)
```

```
def register_vizdoom_components():
    register_vizdoom_envs()
    register_vizdoom_models()


# parse the command line args and create a config
def parse_vizdoom_cfg(argv=None, evaluation=False):
    parser, _ = parse_sf_args(argv=argv, evaluation=evaluation)
    # parameters specific to Doom envs
    add_doom_env_args(parser)
    # override Doom default values for algo parameters
    doom_override_defaults(parser)
    # second parsing pass yields the final configuration
    final_cfg = parse_full_cfg(parser, argv)
    return final_cfg
```

Now that the setup if complete, we can train the agent. We have chosen here to learn a ViZDoom task called `Health Gathering Supreme`.

The scenario: Health Gathering Supreme



Figure 30: ViZDoom scenario: Health Gathering Supreme [original figure]

The objective of this scenario is to **teach the agent how to survive without knowing what makes it survive**. The Agent know only that **life is precious** and death is bad so **it must learn what prolongs its existence and that its health is connected with survival**.

The map is a rectangle containing walls and with a green, acidic floor which **hurts the player periodically**. Initially there are some medkits spread uniformly over the map. A new medkit falls from the skies every now and then. **Medkits heal some portions of player's health** - to survive, the agent needs to pick them up. The episode finishes after the player's death or on timeout.

Further configuration:

- Living_reward = 1
- 3 available buttons: turn left, turn right, move forward
- 1 available game variable: HEALTH
- death penalty = 100

You can find out more about the scenarios available in ViZDoom here.

There are also a number of more complex scenarios that have been create for ViZDoom, such as the ones detailed on this github page.

9.8 Training the agent

- We're going to train the agent for 4000000 steps. It will take approximately 20min

```
## Start the training, this should take around 15 minutes
register_vizdoom_components()

# The scenario we train on today is health gathering
# other scenarios include "doom_basic", "doom_two_colors_easy", "doom_dm", "doom_dwango5", "
    doom_my_way_home", "doom_deadly_corridor", "doom_defend_the_center", "doom_defend_the_line"
env = "doom_health_gathering_supreme"
cfg = parse_vizdoom_cfg(
    argv=[f"--env={env}", "--num_workers=8", "--num_envs_per_worker=4", "--train_for_env_steps
    =4000000"]
)

status = run_rl(cfg)
```

9.9 Let's take a look at the performance of the trained policy and output a video of the agent.

```
from sample_factory.enjoy import enjoy

cfg = parse_vizdoom_cfg(
    argv=[f"--env={env}", "--num_workers=1", "--save_video", "--no_render", "--max_num_episodes
    =10"], evaluation=True
)
status = enjoy(cfg)
```

9.10 Now lets visualize the performance of the agent

```
from base64 import b64encode
from IPython.display import HTML

mp4 = open("/content/train_dir/default_experiment/replay.mp4", "rb").read()
data_url = "data:video/mp4; base64," + b64encode(mp4).decode()
HTML(
    """
<video width=640 controls>
      <source src="%s" type="video/mp4">
</video>
"""
    % data_url
)
```

The agent has learned something, but its performance could be better. We would clearly need to train for longer. But let's upload this model to the Hub.

9.11 Now lets upload your checkpoint and video to the Hugging Face Hub

To be able to share your model with the community there are three more steps to follow:

1. (If it's not already done) create an account to HF -> https://huggingface.co/join
2. Sign in and get your authentication token from the Hugging Face website.

- Create a new token (https://huggingface.co/settings/tokens) **with write role**

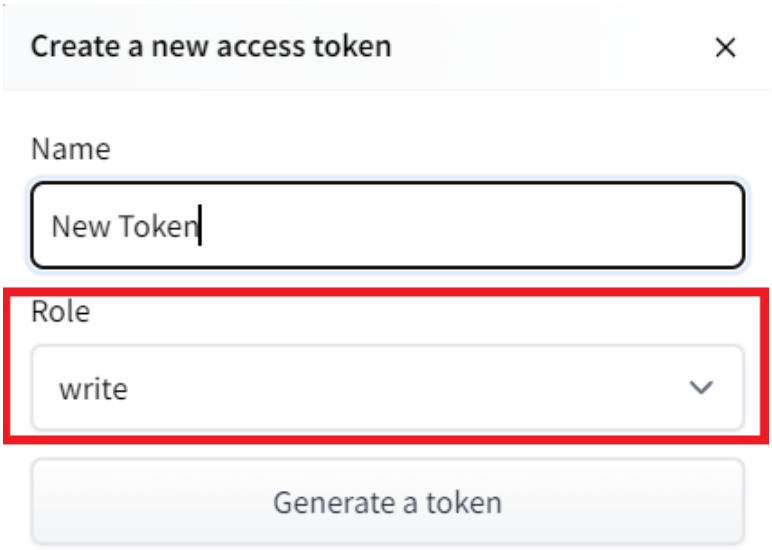


Figure 31: Creating a Hugging Face write-access token [original figure]

- Copy the token
- Run the cell below and paste the token

If you don't want to use Google Colab or a Jupyter Notebook, you need to use this command instead: `huggingface-cli login`

```
from huggingface_hub import notebook_login
notebook_login()
!git config --global credential.helper store
```

```
from sample_factory.enjoy import enjoy

hf_username = "ThomasSimonini"  # insert your HuggingFace username here

cfg = parse_vizdoom_cfg(
    argv=[
        f"--env={env}",
        "--num_workers=1",
        "--save_video",
        "--no_render",
        "--max_num_episodes=10",
        "--max_num_frames=100000",
        "--push_to_hub",
        f"--hf_repository={hf_username}/rl_course_vizdoom_health_gathering_supreme",
    ],
    evaluation=True,
)
status = enjoy(cfg)
```

9.12 Let's load another model

This agent's performance was good, but we can do better! Let's download and visualize an agent trained for 10B timesteps from the hub.

```
#download the agent from the hub
python -m sample_factory.huggingface.load_from_hub -r edbeeching/doom_health_gathering_supreme_2222
     -d ./train_dir
```

```
ls train_dir/doom_health_gathering_supreme_2222
```

```
env = "doom_health_gathering_supreme"
cfg = parse_vizdoom_cfg(
    argv=[
        f"--env={env}",
        "--num_workers=1",
        "--save_video",
        "--no_render",
        "--max_num_episodes=10",
        "--experiment=doom_health_gathering_supreme_2222",
        "--train_dir=train_dir",
    ],
    evaluation=True,
)
status = enjoy(cfg)
```

```
mp4 = open("/content/train_dir/doom_health_gathering_supreme_2222/replay.mp4", "rb").read()
data_url = "data:video/mp4; base64," + b64encode(mp4).decode()
HTML(
    """
<video width=640 controls>
      <source src="%s" type="video/mp4">
</video>
"""
    % data_url
)
```

9.13 Some additional challenges : Doom Deathmatch

Training an agent to play a Doom deathmatch **takes many hours on a more beefy machine than is available in Colab**.

Fortunately, we have have **already trained an agent in this scenario and it is available in the Hub!** Let's download the model and visualize the agent's performance.

```
# Download the agent from the hub
python -m sample_factory.huggingface.load_from_hub -r edbeeching/doom_deathmatch_bots_2222 -d ./
    train_dir
```

Given the agent plays for a long time the video generation can take **10 minutes**.

```
from sample_factory.enjoy import enjoy

register_vizdoom_components()
env = "doom_deathmatch_bots"
cfg = parse_vizdoom_cfg(
    argv=[
        f"--env={env}",
        "--num_workers=1",
        "--save_video",
        "--no_render",
        "--max_num_episodes=1",
        "--experiment=doom_deathmatch_bots_2222",
        "--train_dir=train_dir",
    ],
    evaluation=True,
)
status = enjoy(cfg)
mp4 = open("/content/train_dir/doom_deathmatch_bots_2222/replay.mp4", "rb").read()
```

```
data_url = "data:video/mp4; base64," + b64encode(mp4).decode()
HTML(
    """
<video width=640 controls>
      <source src="%s" type="video/mp4">
</video>
"""
    % data_url
)
```

You **can try to train your agent in this environment** using the code above, but not on colab. **Good luck**

If you prefer an easier scenario, **why not try training in another ViZDoom scenario such as `doom_deadly_corridor` or `doom_defend_the_center`.**

This concludes the last unit. But we are not finished yet! The following **bonus section include some of the most interesting, advanced, and cutting edge work in Deep Reinforcement Learning**.

9.14 Keep learning, stay awesome

10 Conclusion

That's all for today. Congrats on finishing this Unit and the tutorial!

Now that you've successfully trained your Doom agent, why not try deathmatch? Remember, that's a much more complex level than the one you've just trained, **but it's a nice experiment and I advise you to try it.**

If you do it, don't hesitate to share your model in the `#rl-i-made-this` channel in our discord server.

This concludes the last unit, but we are not finished yet! The following **bonus unit includes some of the most interesting, advanced, and cutting edge work in Deep Reinforcement Learning**.

See you next time

10.1 Keep Learning, Stay awesome